CHALLENGE 1: FAKE NEWS / DESINFORMAÇÃO

Relatório técnico do modelo de inteligência artificial VerificAI

Arquitetura, avaliação, limitações e rastreabilidade

Versão 1.0 | 15 de outubro de 2026

Resumo executivo

O VerificAI analisa notícias a partir de uma URL e produz uma estimativa de confiabilidade baseada no texto extraído. A arquitetura atual combina dois classificadores: um ramo léxico com TF-IDF e regressão logística calibrada e um ramo semântico com mDeBERTa-v3 ajustado para notícias. As probabilidades são combinadas e recebem uma calibração adicional antes de serem apresentadas ao usuário.

O experimento do ensemble utilizou 8.295 textos, distribuídos entre treinamento, validação, calibração e teste. No teste reservado de 1.659 registros, o modelo combinado obteve acurácia de **88,49%**, F1 macro de **0,8814** e ROC AUC de **0,9606**. O ganho de F1 macro sobre o mDeBERTa isolado foi de aproximadamente 0,0041; não há análise de significância estatística registrada para essa diferença.

O modelo também apresenta erros relevantes: no mesmo teste, 124 de 716 notícias rotuladas como confiáveis foram classificadas como falsas no limiar binário. A ferramenta deve, portanto, ser interpretada como apoio à análise textual. A versão atual não verifica fatos por consulta a fontes independentes e não está certificada para decisões autônomas sobre a veracidade de notícias.

Este relatório consolida código, manifestos, artefatos e registros locais de experimentos. As métricas de qualidade são resultados históricos conferidos nos arquivos; não foi realizado novo treinamento nem nova avaliação estatística do corpus durante sua elaboração. As verificações de software executadas nesta data estão descritas na seção 9.

1 Objetivo, entrada e saída

O objetivo técnico é estimar a classe de uma notícia a partir de padrões lexicais e contextuais de seu título e corpo. O treinamento é binário, com os rótulos canônicos `0 = falsa` e `1 = confiável`. "Inconclusiva" é uma faixa de decisão aplicada depois da inferência, e não uma terceira classe aprendida.

A aplicação recebe uma URL em `POST /api/analyze`. O serviço de extração usa Requests, Trafilatura e BeautifulSoup para obter título, corpo, autoria, data e domínio. A classificação recebe título e corpo; metadados e cinco critérios heurísticos são apresentados como informações auxiliares. No modo ensemble, esses critérios não entram na pontuação.

Saída	Definição
`prob_fake`	Probabilidade final estimada da classe falsa, entre 0 e 1
`confidence_score`	Pontuação de confiabilidade: `100 * (1 - prob_fake)`
`faixa`	Verdadeira, inconclusiva ou falsa, conforme os limiares do manifesto
`component_probabilities`	Probabilidades dos ramos léxico e semântico antes da calibração final da combinação
`explicacao.local`	Comparações locais obtidas pela remoção de trechos
`provenance`	Hash da entrada, hash do modelo e metadados da extração

O nome "probabilidade de falsidade" identifica a saída estatística do classificador. Não representa uma comprovação factual nem a probabilidade de acerto global do sistema para uma URL específica.

2 Arquitetura e processamento

O fluxo executado é: extração do título e corpo; inferência dos dois ramos em paralelo; média ponderada das probabilidades; calibração final; cálculo da pontuação e faixa; geração da explicação local; retorno pela API.

2.1 Ramo léxico: TF-IDF e regressão logística

O componente importado é `classificador.joblib`, versão `texto-5c379316b7da-20261013T123124`. Ele mantém o pipeline e a calibração de seu treinamento original.

O pré-processamento combina título e corpo, remove marcação HTML residual e conteúdo de scripts e estilos, aplica normalização Unicode NFC, converte para minúsculas e normaliza espaços. Números, acentos e negações são preservados nessa preparação.

Elemento	Configuração
TF-IDF de palavras	N-gramas de 1 a 2; até 20.000 atributos
TF-IDF de caracteres	N-gramas de 3 a 5; até 30.000 atributos
Classificador	Regressão logística, `C=2`, `max_iter=2000`, seed 42
Calibração própria	Sigmoide ajustada no protocolo original
Contração das probabilidades	Aproximação de 0,5 com `r=0.9350741002`
Tratamento de texto curto	Redução adicional da confiança abaixo de 33 palavras preparadas

Se `p_cal` é a probabilidade calibrada de confiabilidade desse ramo e `n` é a quantidade de palavras preparadas, o ajuste final pode ser expresso por:

```
f = min(1, n / 33)
P_lexico(confiavel) = 0.5 + (p_cal - 0.5) * r * f
```

O código também aplica proteção numérica contra extremos exatos. O limite de 33 palavras foi registrado como o percentil 5 do conjunto de treino original. Esse ajuste pertence ao ramo léxico; a calibração posterior do ensemble pode produzir uma estimativa final mais distante de 0,5.

O artefato histórico foi treinado com corpos já transformados e inclui reportagens de checagem rotuladas conforme a alegação discutida. Seu manifesto registra F1 macro de validação cruzada de 0,9613, mas essa métrica pertence a outro corpus e protocolo e não substitui a avaliação do ensemble apresentada neste relatório.

2.2 Ramo semântico: mDeBERTa-v3

O ramo semântico foi inicializado a partir do checkpoint `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli`, na revisão registrada `8adb042d524ecd5c26d3e3ba0e3fbcf7e2d0864c`. A cabeça original de três classes foi substituída por uma cabeça binária, ajustada de forma supervisionada para a tarefa do projeto.

A configuração salva contém 12 camadas, dimensão oculta 768 e 12 cabeças de atenção. O registro de treinamento informa 278.810.882 parâmetros totais, dos quais 14.767.874 eram treináveis, aproximadamente 5,30%. Os embeddings e as primeiras dez camadas do encoder foram congelados; as últimas duas camadas, o pooler e a cabeça binária foram ajustados.

O título e o corpo são concatenados e tokenizados com os arquivos do checkpoint. O modelo usa os primeiros **256 tokens, incluindo tokens especiais**. A configuração da arquitetura admite 512 posições, mas a execução desta versão limita a entrada a 256. O corpo completo permanece disponível ao ramo léxico. A API informa o número de tokens e a ocorrência de truncamento.

Na inferência, o modelo opera em modo de avaliação, sem cálculo de gradientes. Um bloqueio serializa o acesso ao Transformer no mesmo processo. Seus logits são divididos pela temperatura semântica `1.6478583044` antes do softmax.

3 Dados e protocolo experimental

O corpus do experimento combina textos originais do Fake.Br e registros de `banco_fake.csv`. O banco contém 1.113 registros; sua rotulagem integral como falsa foi confirmada pelo responsável em 13/10/2026, conforme a procedência salva. Não houve nova checagem factual automatizada desses rótulos na preparação.

Após a remoção de 18 duplicatas, o conjunto resultante contém:

Classe	Registros
Falsa	4.713
Confiável	3.582
Total	8.295

A preparação remove duplicatas de texto normalizado e de URL. Pares relacionados do Fake.Br compartilham um grupo; registros do banco são agrupados por URL. O particionamento usa `StratifiedGroupKFold`, seed 42, e verifica que os grupos não se cruzem entre as partições do novo experimento.

Partição	Registros	Finalidade
Treinamento	4.976	Ajustar os parâmetros treináveis do Transformer
Validação	829	Selecionar checkpoint e pesos da combinação
Calibração	831	Ajustar temperaturas e intercepto
Teste	1.659	Medir o desempenho da configuração selecionada

Essa organização reduz sobreposição direta entre grupos, mas não exclui todas as paráfrases nem constitui separação completa por publicador ou período. Também não foi certificada a independência entre os dados históricos de treinamento do classificador léxico importado e o corpus do novo experimento. Por isso, o teste deve ser descrito como uma avaliação experimental com essas restrições.

4 Treinamento, combinação e calibração

4.1 Hiperparâmetros do ajuste semântico

Os valores abaixo foram conferidos nos parâmetros do run de treinamento do MLflow e nos metadados do checkpoint.

Parâmetro	Valor
Épocas	2
Batch por dispositivo	4
Acumulação de gradiente	2 passos
Batch efetivo em um dispositivo	8 exemplos
Learning rate do encoder ajustado	`2e-5`
Learning rate do pooler e cabeça	`1e-3`
Otimizador implementado	AdamW com grupos de parâmetros
Weight decay	`0.01`
Warmup	10%
Seed	42
Seleção do checkpoint	Maior F1 macro na validação
Dispositivo do experimento documentado	Apple GPU via MPS

4.2 Seleção da combinação

O experimento comparou cinco pesos para o ramo léxico: 0,10; 0,25; 0,50; 0,75; 0,90. O peso semântico é o complemento de cada candidato. Para cada combinação, temperatura e intercepto foram ajustados na partição de calibração. A escolha ocorreu pelo maior F1 macro na validação, com menor log loss como desempate. O teste foi utilizado posteriormente para medir a configuração selecionada.

O resultado escolhido foi **90% para a probabilidade léxica e 10% para a semântica**, seguido de calibração com temperatura e intercepto. Esses pesos não representam importância causal nem significam que 90% das decisões possam ser atribuídas ao ramo léxico.

Se `pL` e `pS` são as probabilidades de falsidade dos componentes, a forma binária da combinação é:

```
q = 0.9 * pL + 0.1 * pS
z = ln(q / (1 - q)) / T + b
P_final(falsa) = 1 / (1 + exp(-z))

T = 0.24809465108717993
b = 9.835101001562354
```

O código implementa a expressão equivalente com softmax e proteção numérica. O intercepto positivo aumenta a log-odds da classe falsa em relação à combinação sem esse intercepto. Portanto, a probabilidade final pode diferir bastante da média ponderada dos componentes.

A calibração foi aprendida para compensar o deslocamento observado no corpus do experimento. A magnitude do intercepto, isoladamente, não comprova um defeito, mas torna especialmente relevante avaliar a estabilidade das probabilidades em outras fontes, épocas e comprimentos de texto.

4.3 Política de apresentação

Probabilidade final da classe falsa	Apresentação
Até 0,35	Provavelmente verdadeira
Acima de 0,35 e até 0,65	Resultado inconclusivo
Acima de 0,65	Possível desinformação

Esses limiares são uma política do projeto. As métricas binárias da seção seguinte usam a probabilidade da classe confiável igual ou superior a 0,5 para a classe confiável e incluem todos os exemplos, inclusive os que ficam na faixa inconclusiva da interface.

5 Resultados e análise dos erros

5.1 Comparação no teste reservado

Os três componentes foram avaliados nos mesmos 1.659 registros. Os valores abaixo foram extraídos de `evaluation.splits.test` do manifesto e arredondados a quatro casas decimais.

Métrica	Léxico importado	mDeBERTa calibrado	Ensemble calibrado
Acurácia	0,4623	0,8807	0,8849
Precisão macro	0,7226	0,8818	0,8872
Recall macro	0,5270	0,8743	0,8779
F1 macro	0,3594	0,8773	0,8814
ROC AUC	0,9118	0,9531	0,9606
Log loss	1,5709	0,2781	0,2715
Brier score	0,4624	0,0866	0,0827

Acurácia mede a proporção de classificações corretas. Precisão, recall e F1 macro calculam a média simples entre as duas classes. ROC AUC mede a ordenação das previsões usando a classe confiável como positiva no código. Log loss e Brier avaliam a qualidade das probabilidades e são melhores quando menores. ROC AUC de 0,9606 não significa acurácia de 96,06%, e Brier de 0,0827 não representa uma taxa de erro de 8,27%.

O ensemble supera o Transformer em aproximadamente 0,41 ponto percentual de F1 macro na escala de 0 a 100. O resultado é um ganho pontual pequeno, sem intervalos de confiança ou testes estatísticos registrados. O baixo F1 do ramo léxico, apesar de ROC AUC relativamente alto, evidencia a diferença entre ordenar exemplos e acertar decisões no limiar escolhido.

5.2 Matriz de confusão e falsas acusações

A matriz abaixo foi recuperada do artefato `evaluation/deep_ensemble_test.json` do run de seleção/calibração. As linhas são as classes reais; as colunas são as predições binárias.

Classe real	Predita falsa	Predita confiável	Total
Falsa	876	67	943
Confiável	124	592	716
Total	1.000	659	1.659

Considerando falsa como classe positiva, os cálculos derivados dessa matriz são:

- **Taxa de falsas acusações:** `124 / 716` = 17,32% das notícias confiáveis foram classificadas como falsas.
- **Taxa de falsas não detectadas:** `67 / 943` = 7,10% das notícias falsas foram classificadas como confiáveis.
- **Precisão da classe falsa:** `876 / 1.000` = 87,60%.
- **Recall da classe falsa:** `876 / 943` = 92,90%.

Esses números confirmam a ocorrência de falsos positivos no conjunto de teste. Não são taxas da faixa "possível desinformação" da interface, que usa limiar de 0,65. Para conhecer a taxa de falsas acusações nessa faixa, é necessário reavaliar as probabilidades individuais com os limiares da interface; a matriz binária não permite recuperar esse resultado.

5.3 Recorte de notícias falsas do banco

O teste contém 225 registros originados de `banco_fake.csv`, todos rotulados como falsos.

Indicador do ensemble	**Resultado**
Recall no cálculo específico `P(falsa) >= 0.5`	84,89% — 191/225
Faixa falsa, `P(falsa) > 0.65`	80,89% — 182/225
Faixa inconclusiva	8,00% — 18/225

Esse recorte mede a detecção de notícias falsas, mas não permite calcular a taxa de falsas acusações por não conter a classe confiável. Exatamente em 0,5, o cálculo específico desse recorte atribui a classe falsa, enquanto a decisão global atribui a classe confiável; a convenção deve ser preservada ao reproduzir os números.

6 Explicabilidade e relação com a probabilidade

A funcionalidade apresentada como IA explicativa usa o método `sentence_deletion`, que remove frases. Não se trata de um modelo generativo separado e não utiliza SHAP. A previsão original é calculada primeiro; em seguida, variantes do título e corpo são avaliadas pelo mesmo preditor.

Por padrão, até oito trechos são comparados, com limite configurável de dezesseis. O título é candidato quando tem pelo menos três palavras e há corpo; frases do corpo precisam ter pelo menos quatro palavras. A seleção prioriza o título e distribui os demais candidatos pelo texto. O ensemble processa as variantes em lotes de dois.

O efeito de um trecho é calculado em pontos percentuais:

```
efeito = 100 * [P(falsa no texto original) - P(falsa sem o trecho)]
```

Um efeito positivo indica que remover o trecho reduziu a estimativa de falsidade. Um efeito negativo indica o sentido inverso. São destacados até três trechos de cada direção, com magnitude mínima de 0,5 ponto percentual. O resumo é construído a partir dessas comparações medidas.

A explicação é acrescentada após o cálculo da classificação e não participa da fórmula da probabilidade original. Falhas nessa etapa retornam explicação indisponível, preservando a classificação calculada. As contribuições de palavras da regressão logística permanecem separadas e explicam somente o ramo léxico.

Os efeitos não são somáveis nem demonstram causalidade factual. A remoção altera contexto e comprimento e pode trazer novos tokens para a janela de 256 posições do Transformer. Uma

frase pode influenciar a previsão por padrões aprendidos no corpus sem fornecer evidência real de falsidade.

Uma divergência observada após a inclusão da explicação exige comparar a mesma entrada, versão e configuração. A investigação deve registrar título e corpo extraídos, `input_sha256`, backend ativo, versão e hash dos artefatos, probabilidades dos componentes e saída final com a explicação ligada e desligada. A verificação pontual realizada nesta sessão preservou as probabilidades nessas duas condições; isso não identifica a causa de uma URL cujo conteúdo ainda não foi reproduzido.

7 Integração, distribuição e rastreabilidade

O backend Flask serve a interface e os endpoints de análise, estado e recarregamento. Os modos disponíveis são `ensemble`, `calibrated` e `legacy`. Na ausência de configuração explícita, a existência do manifesto do ensemble determina sua seleção padrão. Isso deve ser considerado ao comparar resultados entre ambientes.

O bundle contém manifesto, procedência, classificador importado, pesos do Transformer e arquivos do tokenizador. O inventário registra 11 arquivos necessários, com tamanho total aproximado de 1,06 GiB. Pesos e tokenizadores grandes são distribuídos por Git LFS. A inferência carrega os modelos localmente; a coleta de uma notícia por URL requer acesso à página externa.

O carregamento verifica rótulos, limites, compatibilidade do pré-processamento e versão do scikit-learn no artefato léxico. O ensemble verifica o hash do componente léxico e a assinatura de partições do Transformer. A conferência completa de SHA-256 é feita pelo script de integridade.

Identificador	**Valor**
Run de ajuste semântico	`658901685b1e42f88a0957f73a8e12e9`
Run de seleção/calibração	`8ab1f341b2604e19a894cfd209ca20e2`
Dataset do ensemble — SHA-256	`6789c846eb1b264f71a377bfd818635734540868007d56f6 61823b9d42091e0d`
Classificador importado — SHA-256	`a9270ab5507fbcf3298dc1117f3c6806d36b150a791edc78 65fd4873fac89064`
Assinatura das partições	`abb9fd42cee195ac7f4fabf713f508e3a70a6184703608c3 4296e6c53fffdcd4`

O ambiente documentado usa Python 3.14.6, scikit-learn 1.8.0, PyTorch 2.14.1 e Transformers 4.57.6 no macOS arm64. A disponibilidade de CUDA, MPS ou CPU depende do ambiente. Não há benchmark padronizado de latência e memória que permita estabelecer requisitos de atendimento em produção.

A API registra hashes e metadados de coleta, mas não persiste automaticamente o título e o corpo completos. Assim, um hash permite verificar igualdade, porém não permite recuperar uma entrada antiga caso a página mude.

8 Limitações e melhorias prioritárias

Limitação observada	Consequência	Próxima validação proposta
Falsos positivos no teste	Notícias confiáveis podem ser apresentadas como suspeitas	Avaliar erros por classe e pela política de três faixas, com revisão dos casos
Diferenças entre dados históricos e notícias atuais	Probabilidades podem se comportar de forma distinta fora do corpus	Avaliação externa por fonte e período, com rótulos revisados
Independência histórica não certificada	Possível sobreposição entre treino importado e avaliação	Auditoria dos dados de ambos os componentes
Truncamento semântico em 256 tokens	Partes relevantes do artigo podem ficar fora desse ramo	Avaliar erros por comprimento e alternativas para artigos longos
Rótulos e pistas editoriais do corpus	O modelo pode aprender estilo ou origem em vez da alegação	Revisar rótulos, reportagens de checagem e pistas de classe
Extração variável por página	Mudanças na coleta podem alterar a entrada do modelo	Persistir snapshots e criar reprodução de análises
Avaliação probabilística incompleta	Log loss e Brier não bastam para demonstrar calibração em novas fontes	Curvas de confiabilidade, ECE e análise por subgrupo
Ganho pequeno do ensemble	Não há evidência de superioridade consistente sobre o Transformer	Comparação pareada com incerteza estimada por grupos
Explicação baseada em perturbações	Destaques podem ser confundidos com evidências factuais	Avaliação humana de utilidade e fidelidade das explicações

Essas ações são propostas para versões futuras. O estado atual permanece experimental. Alterações de pesos, calibração ou limiares devem ser escolhidas em dados de desenvolvimento e avaliadas em um conjunto externo reservado, evitando ajustar o sistema diretamente aos erros do teste já consultado.

9 Verificações realizadas para este relatório

Em 15/10/2026 foram executadas as verificações abaixo no ambiente local do projeto:

Verificação	Resultado
Suíte Python completa em modo offline	69 testes aprovados; 2 avisos de bibliotecas
Testes existentes da interface de explicabilidade	2 testes aprovados
Integridade do bundle	11 arquivos presentes; todos os SHA-256 conferidos
Conferência documental	Métricas, matriz de confusão, partições e hiperparâmetros consultados nos artefatos locais

Os avisos da suíte tratam de compatibilidade futura de `torch.jit.script` com Python 3.14 e de uma estratégia de carregamento descontinuada no SQLAlchemy utilizada pelo MLflow. Não houve falha de teste nessa execução.

Comandos de reprodução, a partir da raiz do projeto:

```
HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 .venv-classificador/bin/python -m pytest -q
node --test tests/test_frontend_xai.cjs
.venv-classificador/bin/python scripts/check_model.py --verify-hashes
```

Essas verificações cobrem funcionamento e integridade do software. Não medem a acurácia em novas notícias, não substituem avaliação factual independente e não constituem certificação de prontidão para produção.

10 Conclusão técnica

O VerificAI dispõe de um pipeline integrado de classificação textual, combinação calibrada de modelos, explicações locais e rastreabilidade de experimentos. Os resultados registrados sustentam seu uso como protótipo de pesquisa e apoio à leitura crítica, dentro das condições do corpus avaliado.

A principal limitação para uso mais amplo é a generalização: há falsos positivos mensuráveis, diferenças entre as distribuições dos dados, possível sobreposição histórica e ausência de validação externa consolidada. A prioridade técnica é ampliar essa avaliação, documentar os erros por classe e fonte e tornar cada análise reproduzível antes de promover uma nova versão para produção.

11 Fontes técnicas e evidências

Os dados deste relatório foram obtidos no repositório e no histórico local de experimentos. A cópia das evidências numéricas, `relatorio-tecnico-modelo.evidencias.json`, preserva os resultados e os hiperparâmetros utilizados, permitindo consultar a matriz de confusão mesmo sem restaurar o MLflow.

1. Manifesto do ensemble: `../models/mdeberta-ensemble/manifest.json`; versão, pesos, calibração, métricas e limitações.
2. Procedência dos dados: `../models/mdeberta-ensemble/data_provenance.json`; origem, hashes, classes e partições.
3. Metadados do Transformer: `../models/mdeberta-ensemble/transformer/training_metadata.json`; configuração: `../models/mdeberta-ensemble/transformer/config.json`.
4. Classificador importado: `../models/mdeberta-ensemble/classificador.joblib`; manifesto interno, configuração TF-IDF, calibração e ajuste de texto curto.
5. Preparação de texto: `../ml/preprocessing.py`; artefato léxico: `../ml/artifact.py`; motor original: `../ml/inference.py`.
6. Treinamento semântico: `../backend/ml/deep_training.py`; rotinas de treinamento: `../backend/ml/training.py`; seleção/calibração: `../backend/ml/deep_calibration.py`; inferência do ensemble: `../backend/ml/deep_ensemble.py`.
7. Serviço de análise: `../backend/services/analyzer.py`; extrator: `../backend/services/scraper.py`; explicador local: `../backend/services/explainer.py`.
8. Relatório de teste do MLflow: `../mlruns/3/8ab1f341b2604e19a894cfd209ca20e2/artifacts/evaluation/deep_ensemble_test.json`; matriz de confusão e métricas por classe. Esse caminho depende do histórico local; os dados utilizados estão copiados no arquivo de evidências.
9. Documentação das métricas: `metricas-avaliacao.md`; treinamento mDeBERTa: `mdeberta.md`; explicabilidade: `explicabilidade.md`.